LCEL 체인 조립 (실습)

4-1. 파이프로 연결하기 (실습)
4-2. 체인 디버깅 (실습)
4-3. 사내 문의 응답 체인 (실습)

4-1. 파이프로 연결하기 (실습)

> Prompt, Model, Parser를 파이프로 연결해 첫 체인을 만들고 invoke, stream, batch를 실행해봅니다.

왜 파이프로 연결하나

3번 세션에서 세 부품을 따로따로 실행해 봤습니다. 그대로 이어 붙이면 이렇게 됩니다.

```
# 3번 세션 components.py — 부품을 하나씩 손으로 실행했습니다
filled = prompt.invoke({"text": "Good morning", "language": "한국어"})
ai_message = llm.invoke(filled)
text = parser.invoke(ai_message)

# 같은 일을 LCEL로 조립하면
chain = prompt | llm | parser
text = chain.invoke({"text": "Good morning", "language": "한국어"})
```

위와 아래 코드는 같은 일을 합니다. 줄 수가 줄어든 것이 핵심이 아니라, **조립해 둔 `chain` 이라는 물건이 생긴다는 것**이 핵심입니다.

1. 한 번 조립하면 실행 방식을 골라 쓸 수 있습니다.

같은 `chain` 하나를 `invoke` (답을 한 번에 받기), `stream` (생성되는 대로 흘려 받기), `batch` (여러 건을 한꺼번에 처리) 세 가지로 실행할 수 있습니다.
위의 손으로 쓴 3줄에 타이핑 효과(스트리밍)를 넣으려면 모델 호출부터 파서까지 다시 짜야 합니다. 체인은 `chain.stream(...)` 처럼 **메서드 이름만 바꾸면** 됩니다.

2. 부품 교체가 쉬워집니다.

모델을 다른 모델로 바꾸거나 파서를 `JsonOutputParser` 로 갈아 끼워도 나머지 코드는 그대로입니다. 모든 부품이 Runnable이라는 같은 규격을 따르기 때문입니다.

3. 체인 자체가 다시 부품이 됩니다.

조립한 체인도 Runnable이므로 더 큰 체인 안에 넣을 수 있습니다. 2일차 RAG에서 검색 결과를 프롬프트에 끼울 때, 3일차 Agent의 노드 안에서도 오늘 만드는 체인이 그대로 들어갑니다.

위 나열한 이점을 가지고 쓰는게 바로 LCEL입니다.

Without LCEL

```python
from typing import List
import openai

prompt_template = "Tell me a short joke about {topic}"
client = openai.OpenAI()

def call_chat_model(messages: List[dict]) -> str:
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
    )
    return response.choices[0].message.content

def invoke_chain(topic: str) -> str:
    prompt_value = prompt_template.format(topic=topic)
    messages = [{"role": "user", "content": prompt_value}]
    return call_chat_model(messages)

invoke_chain("Computer")
```

LCEL

```python
from langchain.schema.runnable import RunnablePassthrough
prompt = ChatPromptTemplate.from_template
        ("Tell me a short joke about {topic}")
output_parser = StrOutputParser()
model = ChatOpenAI(model_name="gpt-3.5-turbo")
chain = (
    {"topic": RunnablePassthrough()}
    | prompt
    | model
    | output_parser
)
response = chain.invoke("Computer")
print(response)
```

LCEL(LangChain Expression Language, 랭체인 표현식 언어)은 부품들을 파이프 연산자 `|` 로 연결해 체인을 만드는 문법입니다.

이 기호는 유닉스의 `ls | grep` 과 같은 발상에서 나왔다고 하며, 모든 부품이 Runnable 표준 인터페이스를 따릅니다. 그래서 체인을 한 번 만들면 코드를 바꾸지 않고 실행 메서드만 바꿔 세 가지 방식으로 실행할 수 있습니다.

- **단건**: `invoke` 로 한 번 호출하고 한 번 응답받습니다.
- **실시간**: `stream` 으로 결과를 토큰 단위로 받습니다.
- **병렬**: `batch` 로 여러 입력을 한꺼번에 처리합니다.

* 참고로 세 가지 모두 이름 앞에 `a` 를 붙인 **비동기 버전**이 짝으로 있습니다. `ainvoke` , `astream` , `abatch` 입니다. 하는 일은 똑같고 부르는 방식만 다릅니다.
`await chain.ainvoke(...)` 처럼 `await` 를 붙여 부르며 그 코드는 `async def` 로 선언한 함수 안에 있어야 합니다.

교육에서는 각자 스크립트를 돌려보는 상황이며 차이가 크게 드러나지 않으므로 동기로 실습합니다.

4일차에서 MCP 도구를 붙일 때 SDK가 비동기를 요구해서 `ainvoke` 와 `astream` 을 실제로 쓰게 됩니다.

참고로 `batch` 는 내부적으로 이미 여러 호출을 동시에 다룹니다. 그래서 입력 묶음 하나를 빨리 처리하는 것이 목적이라면 동기 `batch` 로 충분하고 서버가 동시에 들어오는 요청을 감당해야 하는 상황이라면 `abatch` 가 필요합니다.

따라하기

1. 체인을 조립하고 세 가지 실행 방식을 비교합니다. 아래 코드를 `day1_practice/lcel_basic.py` 로 저장하고 실행합니다.

```
# lcel_basic.py - 체인 구성과 invoke, stream, batch
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

prompt = PromptTemplate.from_template(
    "너는 삼성SDS 사내 커뮤니케이션 담당자야. {topic}에 대한 공지 문구를 두 문장으로 써줘."
)
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
parser = StrOutputParser()

# 파이프 하나로 조립: dict -> 프롬프트 -> AIMessage -> str
chain = prompt | llm | parser

# 1) invoke: 단건 호출
print("[invoke]")
print(chain.invoke({"topic": "사내 시스템 정기 점검"}))
print("---")

# 2) stream: 토큰 단위 실시간 출력 (챗봇 UI의 타이핑 효과)
print("[stream] ", end="", flush=True)
for chunk in chain.stream({"topic": "보안 교육 일정"}):
    print(chunk, end="", flush=True)
print("\n---")

# 3) batch: 여러 입력을 한꺼번에 병렬 처리
inputs = [
    {"topic": "주차장 공사"},
    {"topic": "구내식당 메뉴 개편"},
    {"topic": "사원증 재발급 안내"},
]
results = chain.batch(inputs)
print("[batch]")
for i, r in zip(inputs, results):
    print(f"- {i['topic']}: {r[:30]}...")
```

참고로 batch 결과가 중간에 끊겨 보이는 것은 응답이 잘린 것이 아니라 화면에 앞 30자만 나오게 코드를 생성해두었습니다. 세 건 모두 온전히 생성돼 `results` 안에 들어 있으니 전문을 보려면 `r[:30]` 을 `r` 로 바꾸면 됩니다.

`stream` 은 전체 응답을 기다리지 않고 생성되는 대로 받아 사용자 체감 속도를 높일 수 있고, `batch` 는 for문으로 `invoke` 를 반복하는 것보다 훨씬 빠르니 사용해보면 좋습니다.

막힐 때

- `TypeError: unsupported operand type(s) for |`

  파이프 양쪽 중 하나가 Runnable이 아닐 때 납니다. `StrOutputParser` 뒤에 괄호를 누락시키는 경우가 흔한 실수 중에 하나입니다.

- stream 출력이 한 번에 몰려 나옵니다.
  `flush=True` 를 빼먹은 경우에 그렇습니다. 해당 옵션을 추가해줍니다.

- batch에서 한 건이 실패하면 전체가 죽습니다.

  `chain.batch(inputs, return_exceptions=True)` 를 쓰면 실패 건은 예외 객체로 받고 나머지는 정상 처리됩니다.

확인: invoke, stream, batch 세 방식이 모두 응답을 내면 성공입니다.

4-2. 체인 디버깅 (실습)

> 중간 단계 입출력을 직접 찍어보며 체인의 데이터 흐름을 추적해봅니다.

체인이 길어지면 중간에 무슨 데이터가 흘렀는지 봐야 문제를 찾을 수 있습니다. 최종 출력이 이상할 때 원인이 프롬프트 완성 단계인지, 모델 응답 단계인지, 파서 단계인지 구분해야 합니다.

그래서 체인 사이에 값을 출력하고 그대로 통과시키는 함수를 끼워 넣는 방식을 씁니다.



여기서 중간에 넣는 함수를 **공항 검색대**라고 생각해보겠습니다. 즉, 비유하자면 공항에서 가방을 검색대에 놓고 X-ray, 검색요원이 각각 검사하는 것과 비슷하다고 생각하시면 될 것 같습니다.

다만 코드에서의 검색대는 지금 흘러온 값을 화면에 찍고 받은 값을 그대로 다음 부품에 넘깁니다. 값에 손을 대지 않으니 결과는 똑같습니다.

그리고 검색대를 여러 군데 놓을 거라 어느 지점에서 찍힌 것인지 구분할 이름표가 필요합니다. `tap("입력")` 처럼 이름표를 주면 그 이름표가 붙은 검색대를 하나 만들어 줍니다. `tap("입력")` 과 `tap("모델 응답 후")` 를 각각 부르면 이름표만 다른 검사대가 두 개 생깁니다.

코드를 보게 보면 `tap` 함수 안에 또 다른 함수 `_tap` 이 들어 있습니다. 파이썬에서는 이렇게 함수가 함수를 만들어 돌려줄 수 있습니다. 지금은 "이름표를 주면 검사대를 하나 만들어 준다" 정도로 읽고 넘어가도 충분합니다.

따라하기

1. 관찰 포인트(검색대)를 끼워 넣은 체인을 실행합니다. 아래 코드를 `day1_practice/lcel_debug.py` 로 저장하고 실행합니다.

```
# lcel_debug.py — 중간 단계 입출력 추적
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

load_dotenv()

def tap(label):
    """값을 출력하고 그대로 다음 단계로 통과시키는 관찰 지점을 만듭니다."""
    def _tap(value):
        print(f"\n=== [{label}] 타입: {type(value).__name__} ===")
        print(value)
        return value  # 반드시 그대로 반환해야 체인이 이어집니다
    return RunnableLambda(_tap)

prompt = PromptTemplate.from_template("{word}의 반대말 하나만 답해.")
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)

# 각 단계 사이에 관찰 지점을 끼워 넣습니다.
chain = (
    tap("입력")
    | prompt
    | tap("프롬프트 완성 후")
    | llm
    | tap("모델 응답 후")
    | StrOutputParser()
)

result = chain.invoke({"word": "출근"})
print("\n최종 결과:", result)
```

전역 로그가 필요하면 다음 두 줄로 켜고 끕니다.

```
from langchain_core.globals import set_debug
set_debug(True)  # 이후 모든 체인 실행의 단계별 입출력이 로그로 출력됩니다
```

막힐 때

- 관찰 포인트를 넣었더니 체인이 `None`을 반환합니다.
  `_tap` 함수에서 `return value`를 빼먹은 경우입니다.
- `set_debug(True)` 로그가 너무 많습니다.
  전역 스위치라 모든 호출에 적용됩니다. 문제 구간만 볼 때는 tap 방식이 낫습니다.

확인: dict → PromptValue → AIMessage → str로 변환되며 흐르는 과정이 출력되면 성공입니다.

> 체인 중간에 흐르는 데이터를 직접 관찰하는 디버깅 방법을 익혔습니다.

4-3. 사내 문의 응답 체인 (실습)

> 오늘 배운 내용을 기반으로 삼성SDS 사내 문의 자동 응답 체인을 완성해봅니다.

직원의 문의 텍스트를 받아 담당 부서를 분류한 뒤 답변 초안을 생성해 구조화된 dict로 반환합니다.

새로 등장하는 부품은 dict 조립입니다. **LCEL에서 여러 체인을 dict로 묶으면 각 값이 병렬로 실행되어 하나의 dict로 합쳐집니다.**

따라하기

1. 분류 체인과 답변 체인을 만들어 병렬로 조립합니다. 아래 코드를 `day1_practice/inquiry_chain.py` 로 저장하고 실행합니다.

```
# inquiry_chain.py - 사내 문의 응답
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import ChatPromptTemplate, PromptTemplate
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.runnables import RunnableParallel

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)

# --- 부품 A: 문의 분류 체인 (구조화 출력) ---
class Routing(BaseModel):
    department: str = Field(description="담당 부서: 'IT지원팀', '인사팀', '총무팀'
중 하나")
    urgency: str = Field(description="긴급도: '높음', '보통', '낮음' 중 하나")

routing_parser = JsonOutputParser(pydantic_object=Routing)
routing_prompt = PromptTemplate(
    template="다음 사내 문의의 담당 부서와 긴급도를 판단해.\n\n문의: {inquiry}\n\n{forma
t_instructions}",
    input_variables=["inquiry"],
    partial_variables={"format_instructions": routing_parser.get_format_inst
ructions()},
)
routing_chain = routing_prompt | llm | routing_parser

# --- 부품 B: 답변 초안 체인 (문자열 출력) ---
answer_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 삼성SDS 사내 헬프데스크 상담원이야. 정중한 존댓말로 답하고 "
     "마지막 문장에 예상 처리 기한을 안내해. 세 문장 이내로 써."),
    ("human", "{inquiry}"),
])
answer_chain = answer_prompt | llm | StrOutputParser()

# --- 조립: 두 체인을 병렬 실행 후 하나의 dict로 합칩니다 ---
full_chain = RunnableParallel(routing=routing_chain, draft=answer_chain)

# --- 실행 ---
inquiry = {"inquiry": "재택근무 신청 절차가 어떻게 되나요? 다음 주부터 필요합니다."}
result = full_chain.invoke(inquiry)
```

```
print("담당 부서:", result["routing"]["department"])
print("긴급도:", result["routing"]["urgency"])
print("답변 초안:")
print(result["draft"])
```

2. 문의 내용을 바꿔가며 분류와 답변이 어떻게 달라지는지 2-3건 더 실행해 봅니다.

막힐 때

- 답변 초안이 아는 척(환각)을 합니다.
  예상 처리 기한 같은 내용은 모델이 지어낸 것입니다. 실제 사내 규정을 근거로 답하게 만드는 것이 2일차의 RAG입니다. 오늘은 형식을 통제한 초안 생성까지가 목표이니 환각은 확인만 하도록 합니다.
- 두 체인이 정말 병렬로 도나요?
  네, 내부적으로 동시에 실행됩니다.

확인: 문의 텍스트 하나로 분류 결과(dict)와 답변 초안(str)이 한 번에 나오면 성공입니다.

> 오늘 배운 부품을 모아 첫 완성 체인을 만드는 경험을 했습니다.
